

Code Explanation: Data Distribution and Visualization

Project: NeuroVision (Brain Tumor Classification) **Objective:** To perform Exploratory Data Analysis (EDA) by visualizing class distributions and inspecting sample images, aligning with the Unit II Lab syllabus.

1. Library Imports and Path Configuration

This section imports the necessary tools and sets up the folder locations.

- `import os`: Imports the Operating System library to interact with files and directories.
- `import matplotlib.pyplot as plt`: Imports the plotting library used to create charts and graphs (like our bar chart and image grid).
- `import matplotlib.image as mpimg`: Imports a specific module to read image files and convert them into mathematical arrays.
- `base_dir = '/content/dataset/archive (1)'`: Defines the main folder where the extracted dataset is stored.
- `yes_path = os.path.join(base_dir, 'yes')`: Dynamically creates the exact file path to the folder containing tumor ('yes') images.
- `no_path = os.path.join(base_dir, 'no')`: Creates the path to the folder containing normal ('no') images.

2. Class Distribution Analysis

This block counts the images to check for balanced or imbalanced data and creates a count plot.

- `yes_images = os.listdir(yes_path)`: Scans the 'yes' folder and creates a list of all the image file names inside it.
- `no_images = os.listdir(no_path)`: Scans the 'no' folder and creates a list of all the image file names inside it.
- `yes_count = len(yes_images)`: Calculates the total number of images in the 'yes' folder by finding the length of the list.
- `no_count = len(no_images)`: Calculates the total number of images in the 'no' folder.
- `print(...)`: Prints the calculated totals to the screen so we can see the exact numbers.
- `plt.figure(figsize=(6, 4))`: Creates a blank canvas for our chart with a width of 6 inches and a height of 4 inches.

- `plt.bar(['Tumor (Yes)', 'Normal (No)'], [yes_count, no_count], color=['red', 'green'])`: Plots a bar chart with two bars representing our classes and their respective counts.
- `plt.title(...)` & `plt.ylabel(...)`: Adds a title to the top of the chart and a label to the Y-axis to make the graph readable.
- `plt.show()`: Renders and displays the final bar chart on the screen.

3. Data Visualization (Displaying Sample Images)

This section selects one image from each class and displays them side-by-side to visually recognize image patterns.

- `sample_yes_img = os.path.join(yes_path, yes_images[0])`: Grabs the very first image file name from the 'yes' list and creates its full path.
- `sample_no_img = os.path.join(no_path, no_images[0])`: Grabs the very first image file name from the 'no' list and creates its full path.
- `img_yes = mpimg.imread(sample_yes_img)`: Reads the tumor image file and converts its grayscale intensity values (pixels) into a numerical format the computer can process.
- `img_no = mpimg.imread(sample_no_img)`: Reads the normal image file and converts it into a numerical array.
- `fig, axes = plt.subplots(1, 2, figsize=(10, 5))`: Creates a 1x2 grid (1 row, 2 columns) to display our two sample images side-by-side.
- `axes[0].imshow(img_yes, cmap='gray')`: Displays the tumor image in the first slot (`axes[0]`). The `cmap='gray'` argument ensures the image is shown in grayscale.
- `axes[0].set_title(...)`: Adds a title exactly above the first image.
- `axes[0].axis('off')`: Hides the standard graph axes (the numbers and borders around the image) for a cleaner, Apple-style visual layout.
- `axes[1].imshow(img_no, cmap='gray')`: Displays the normal image in the second slot (`axes[1]`) in grayscale.
- `axes[1].set_title(...)` & `axes[1].axis('off')`: Adds a title and hides the axes for the second image.
- `plt.tight_layout()`: Automatically adjusts the spacing between the images so they do not overlap.
- `plt.show()`: Renders and displays the final side-by-side image grid.